

Document Intelligence — Architecture & Design

July 17, 2026

CONTENTS

Document Intelligence — Architecture & Design	
1. What this system is	
External dependencies (provided by your org, injected via env vars)	
2. High-level architecture	
Request/middleware order (di/app.py)	
3. Data model	
The knowledge subtree, visually	
Key schema facts (evidence)	
4. The ingest pipeline	
Stage detail	
5. Multi-tenancy & the RLS role split	
6. The durable job queue & workers	
7. What makes it fast	
7.1 OCR & ingest latency avoidance	
7.2 Read & query speed	
7.3 Client-level access & isolation at scale	
7.4 Write / ingest throughput	
8. API surface (reference)	
9. Running everything in one container / pod	
Mode A — single process, embedded worker (simplest)	
Mode B — two processes, one pod (production-posture-clean)	
Required / notable environment variables	
Verification (once the container is up)	
10. MCP endpoint — design (for agents to use the platform)	
10.1 Shape & transport	
10.2 Auth & tenant scoping (identical guarantees to REST)	
10.3 Proposed tools	
10.4 Resources	
10.5 Mounting so “one container” still holds	
10.6 Implementation (shipped)	
10.7 Security considerations	
10.8 Decisions made (changeable on request)	

- 11. Appendix
 - 11.1 Migrations
 - 11.2 Selected configuration (di/config.py)
 - 11.3 Production posture (enforced at boot by di/posture.py)

Document Intelligence — Architecture & Design

A per-client KYC knowledge-tree platform: PII-safe, provenance-tracked, horizontally scalable.

Version	0.2.0 (post enterprise scale-out, migrations 001–011)
Date	2026-07-17
Scope of this document	Current architecture · performance model · single-pod deployment · MCP endpoint design
Audience	Platform engineers, SREs, integrators, and agent developers

Diagrams use [Mermaid](#) fenced blocks. They render natively on GitHub, in VS Code (with the Markdown Preview Mermaid extension), Obsidian, Typora, and most modern Markdown viewers. If your viewer shows raw code blocks, open the file in one of those.

1. What this system is

Document Intelligence ingests KYC documents (passports, national-ID cards, tax forms, incorporation papers, ...) for **millions of independent client tenants** and turns each into a **queryable knowledge tree** with cell-level provenance, PII-safe masking, and a cross-document merged “current view” of each client’s facts.

It is built for three hard requirements a bank imposes:

- 1. **Tenant isolation at scale** — one Postgres cluster, millions of `client_id`s, no tenant can ever see another’s data (enforced in the database, not just the app).
- 2. **PII safety** — a local classifier gate decides, per document, whether any content may leave the perimeter to an LLM; sensitive values are masked by default on the way out.
- 3. **Provenance & auditability** — every extracted fact traces to a source document / page / bounding box / extractor / model, every read of tenant PII is logged, and every reviewer decision survives re-ingestion.

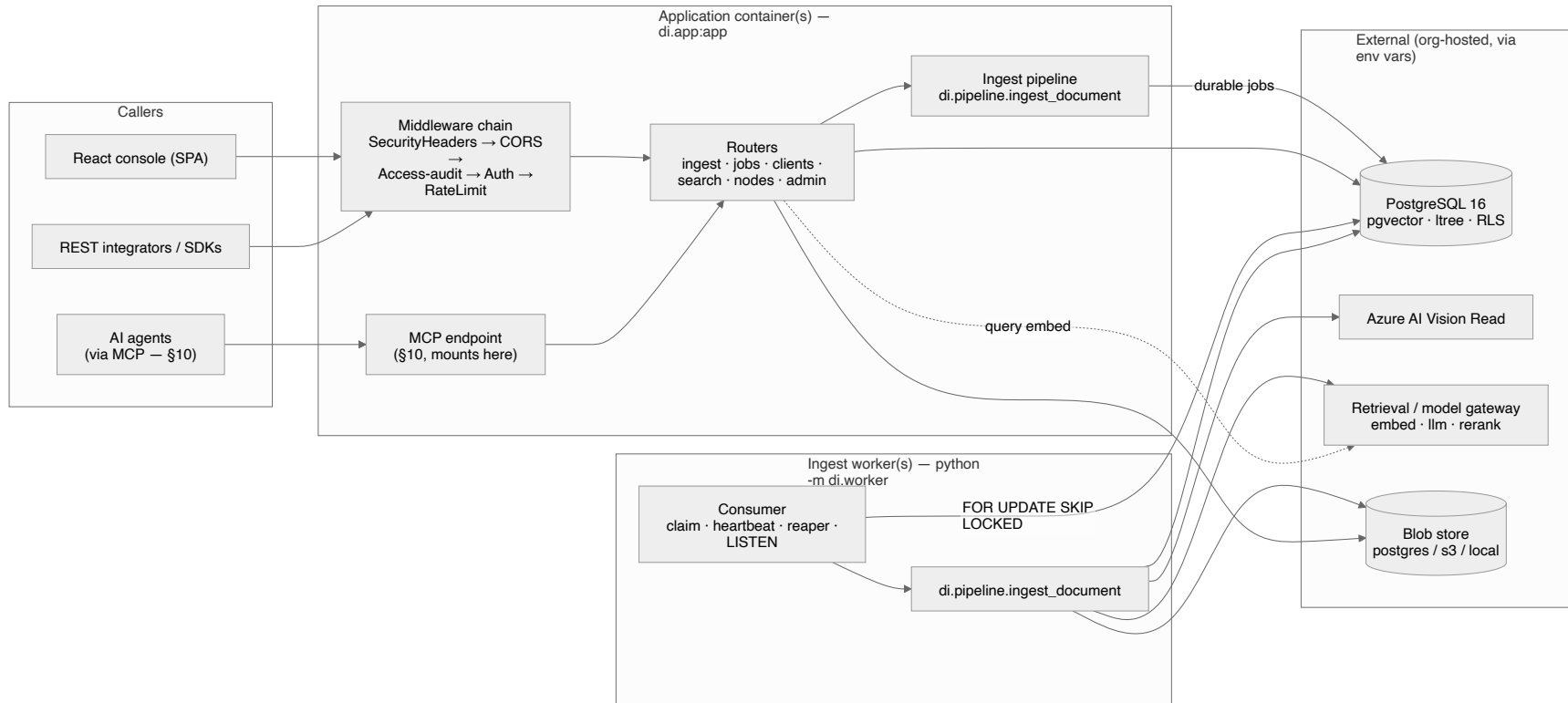
Jurisdictions: US / CA / MX. **Languages:** English / Spanish.

EXTERNAL DEPENDENCIES (PROVIDED BY YOUR ORG, INJECTED VIA ENV VARS)

Dependency	Role	How it’s configured
PostgreSQL 16 + pgvector + ltree	System of record: documents, the knowledge tree, merged facts, the durable job queue, audit log	<code>PG_HOST</code> , <code>PG_PORT</code> , <code>PG_USER</code> , <code>PG_PASSWORD</code> , <code>PG_DATABASE</code> , <code>PG_SCHEMA</code>
Azure AI Vision Read (OCR)	Optical character recognition for scanned images / non-digital PDFs	<code>AZURE_VISION_ENDPOINT</code> , <code>AZURE_VISION_KEY</code>
Retrieval / model gateway	Embeddings + LLM completion + rerank (the platform never calls a model vendor directly)	<code>RETRIEVAL_BASE_URL</code> , <code>RETRIEVAL_API_KEY</code> (or <code>DI_RETRIEVAL_STUB=true</code> offline)
Blob store	Raw uploaded bytes (<code>postgres</code> bytea / <code>s3</code> / <code>local</code> volume / <code>none</code>)	<code>BLOB_BACKEND</code> + backend-specific vars

Postgres and Azure Vision Read are **separate services already hosted in your org** — the application container connects to them purely through the environment variables above. Everything else (API, ingest workers, migrations, the future MCP endpoint) is *application code* that can run in a **single container/pod** (see §9).

2. High-level architecture



Two roles, one codebase. The FastAPI **API** accepts uploads and serves reads; the **worker** drains the durable queue and runs the ingest pipeline. Both import the *same* `di.pipeline.ingest_document`. In a scaled deployment they are separate processes; in a single-pod deployment they collapse into one (§9). The **MCP endpoint** (§10) mounts onto the same FastAPI app so it ships in the same container.

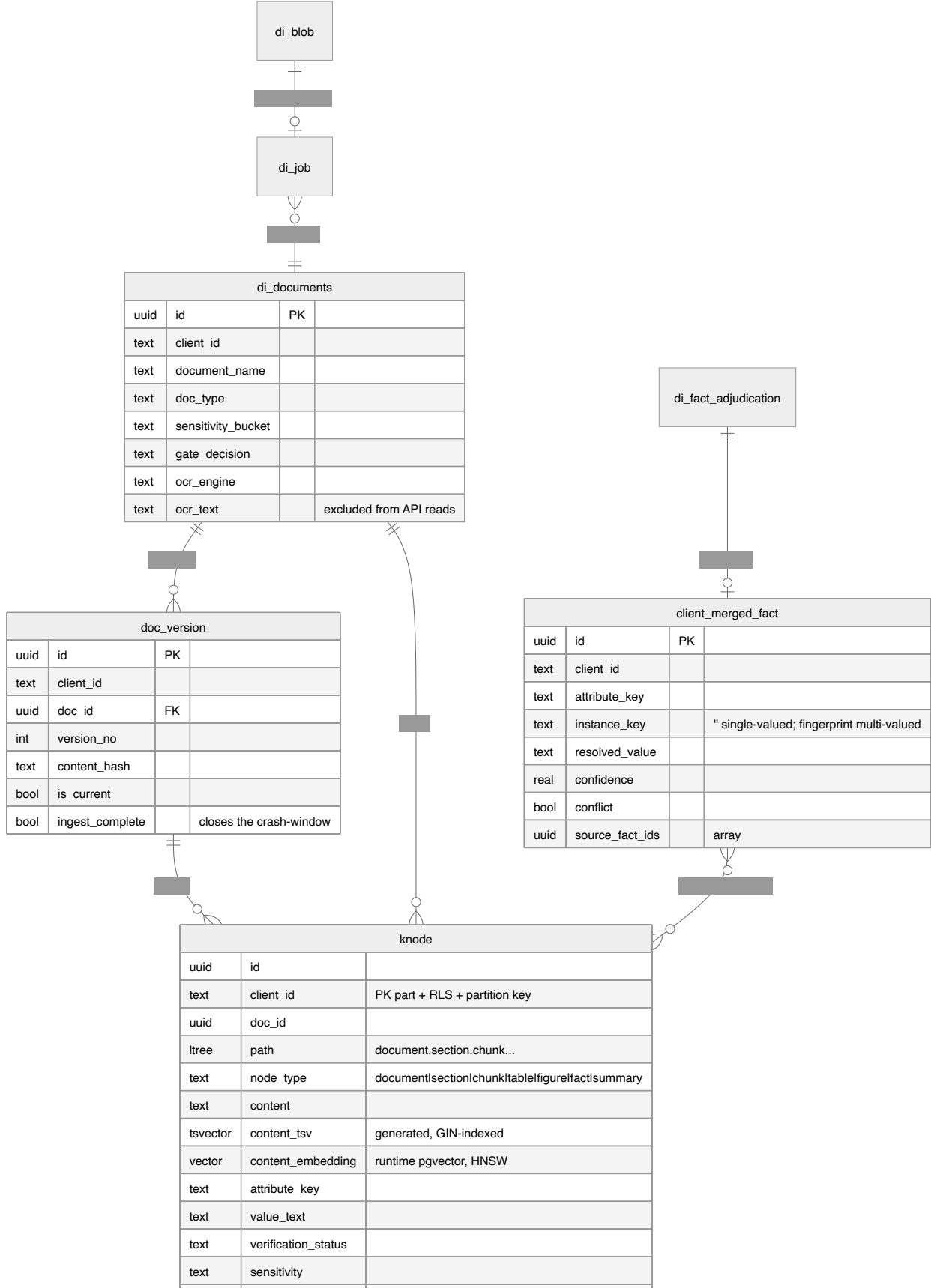
REQUEST/MIDDLEWARE ORDER (DI/APP.PY)

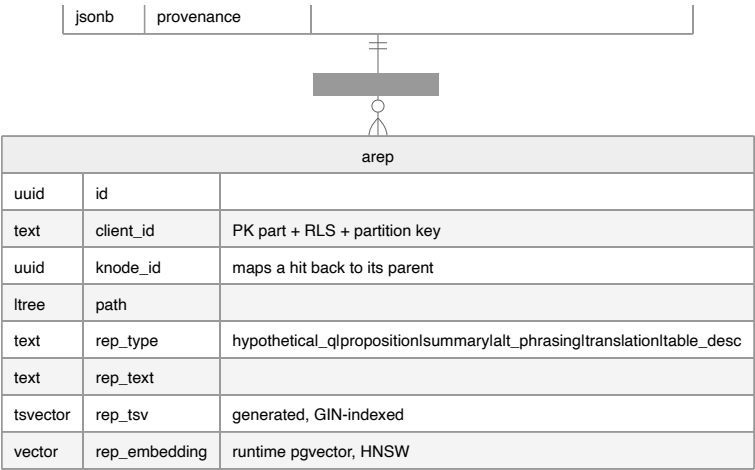
Security headers → CORS → Access-audit middleware → Auth (X-API-Key) → Rate limit → Route

- `GET /health` — pure liveness (`di/app.py:349`).
 - `GET /readyz` — per-dependency truth (db, migrations, RLS posture, pgvector, blob, ocr, auth, audit, queue); returns **503** if any *required* component is down (`di/app.py:354`).
 - `GET /metrics` — Prometheus exposition (`di/app.py:367`).
-

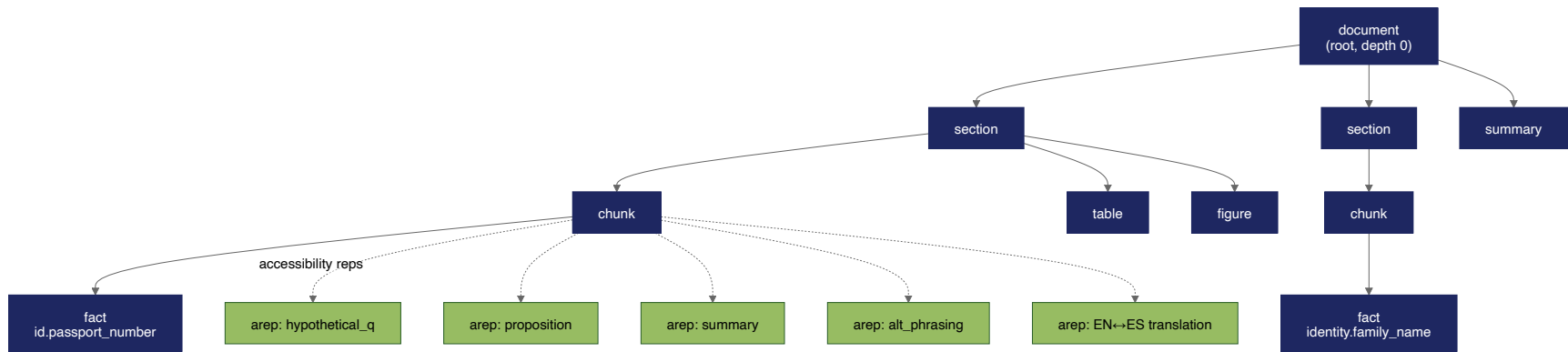
3. Data model

The heart of the system is the **knowledge subtree**: two tables implementing an *index-many / return-parent* retrieval pattern.





THE KNOWLEDGE SUBTREE, VISUALLY



- `knode` rows (blue) are what consumers *get back*. Node types form a tree via an `ltree` `path`: `document` → `section` → `chunk`|`table`|`figure` → `fact` → `summary`.
- `arep` rows (green) are *never returned directly* — they are extra “surfaces” of a chunk (a hypothetical question it answers, a one-sentence proposition, a paraphrase, an EN↔ES translation). Search matches an `arep`, then **returns its parent `knode`**. This is the *index-many / return-parent* pattern: many searchable representations, one canonical answer.

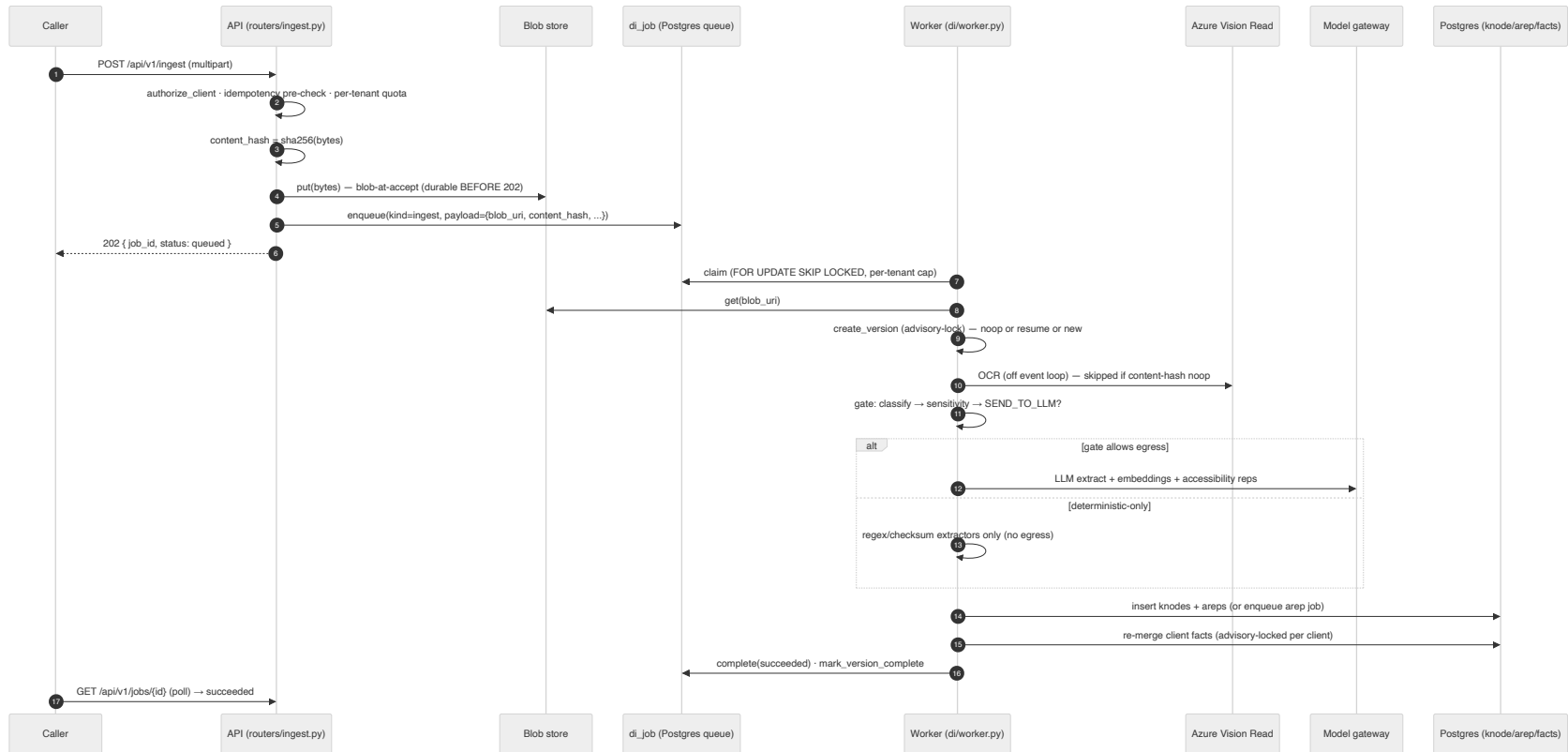
KEY SCHEMA FACTS (EVIDENCE)

- `knode` and `arep` are **HASH-partitioned by `client_id`** (`di/migrations/003_knode_arep.sql:39,63`), **PRIMARY KEY** (`client_id, id`).
- `content_tsv` / `rep_tsv` are **generated `tsvector` columns**, GIN-indexed (`003:20–21,42,59,66`).
- `path` carries a **GiST index** for subtree containment (`003:41,65`).
- `content_embedding` / `rep_embedding` (`vector`) columns + **per-partition HNSW indexes** are added at runtime when `pgvector` is present (`di/db.py:550–572`).
- Every tenant table has **FORCE ROW LEVEL SECURITY** with a `tenant_isolation` policy keyed on `current_setting('app.current_client_id')` (`di/migrations/004_rls.sql`).

4. The ingest pipeline

A document flows `accept` → `durable job` → `OCR` → `gate` → `extract` → `subtree` → `accessibility-reps` → `merge` → `done`. The `accept` path returns `202 Accepted` + a `job_id` immediately; a worker does the heavy work.

Document Intelligence — Architecture & Design

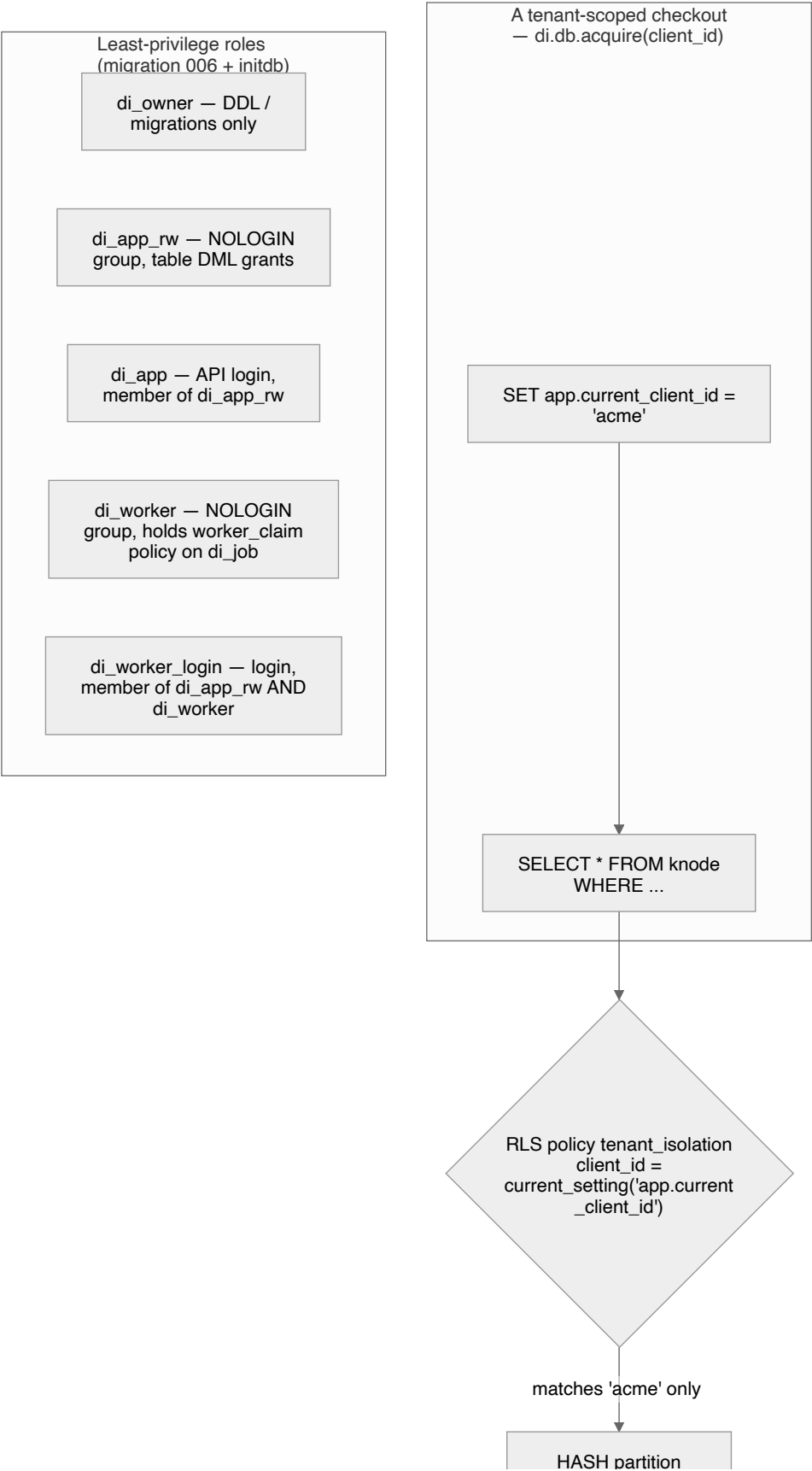


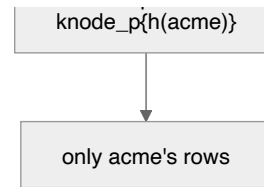
STAGE DETAIL

1. **Accept** (`di/routers/ingest.py`): `authorize_client` → **idempotency pre-check before quota** (a retry of an already-accepted submit must never `429`) → per-tenant admission quota → `content_hash` → **blob-at-accept** (bytes are made durable *before* the 202 and *before* the job row, so a crash between accept and claim can never lose the payload the bank was told was accepted) → `jobs.enqueue` . `BLOB_BACKEND=none` **rejects** async ingest with `503` .
 2. **Claim** (`di/worker.py`): the worker claims with `FOR UPDATE SKIP LOCKED` under a per-tenant running cap (§6).
 3. **Version decision** (`di/store.py:create_version`): under a **per-document advisory lock**, decide *new version* / *noop* (identical content already complete) / *resume* (identical content but a previous attempt crashed mid-pipeline → rebuild). Returns a `VersionResult` that is the single source of version identity for everything downstream.
 4. **OCR** (`di/ocr/vision.py`): engine resolution — **Azure Vision Read** for images/scanned PDFs when configured; **pypdf** text-layer for digital PDFs (free, offline); **tesseract** fallback; **docx** extraction; else empty. OCR **never raises** (degrades). It runs **off the event loop** (`anyio.to_thread`) so a ~60s Azure round-trip never blocks concurrent reads.
 5. **Gate** (`di/gate/`): language profile → classifier (doc type + confidence) → PII sweep → **routing**: `SEND_TO_LLM` (low-sensitivity, confident, gate open) / `REDACT_THEN_SEND` (inactive in v1) / `DETERMINISTIC_ONLY` (anything sensitive or low-confidence). This is the **egress control**: only `SEND_TO_LLM` documents ever reach the model gateway.
 6. **Extract**: deterministic ID extractors always run (passport MRZ, US SSN/EIN/ITIN, CA SIN/BN, MX CURP/RFC/INE — via `python-stdnum` checksums); LLM extraction runs **only** when the gate allowed egress.
 7. **Subtree**: build `knode` rows (sections/chunks/tables/facts/summary), embed content.
 8. **Accessibility reps**: when `AREP_ASYNC=true`, arep generation (5 rep types × N content nodes, each an LLM call) is deferred to a separate **arep queue job** (idempotent per version) instead of blocking the ingest job.
 9. **Merge**: recompute the client's cross-document merged facts (confidence-weighted, multi-valued via `instance_key`), under a **per-client advisory lock** so two workers finishing different docs for one client can't publish a stale merged view.
 10. **Done**: `mark_version_complete` (flips `ingest_complete=true`) then the terminal `done` event.
-

5. Multi-tenancy & the RLS role split

Isolation is enforced in **Postgres**, not merely in the app. Every tenant-scoped query runs on a connection that has bound the tenant GUC; the RLS policy filters rows the query never even names.

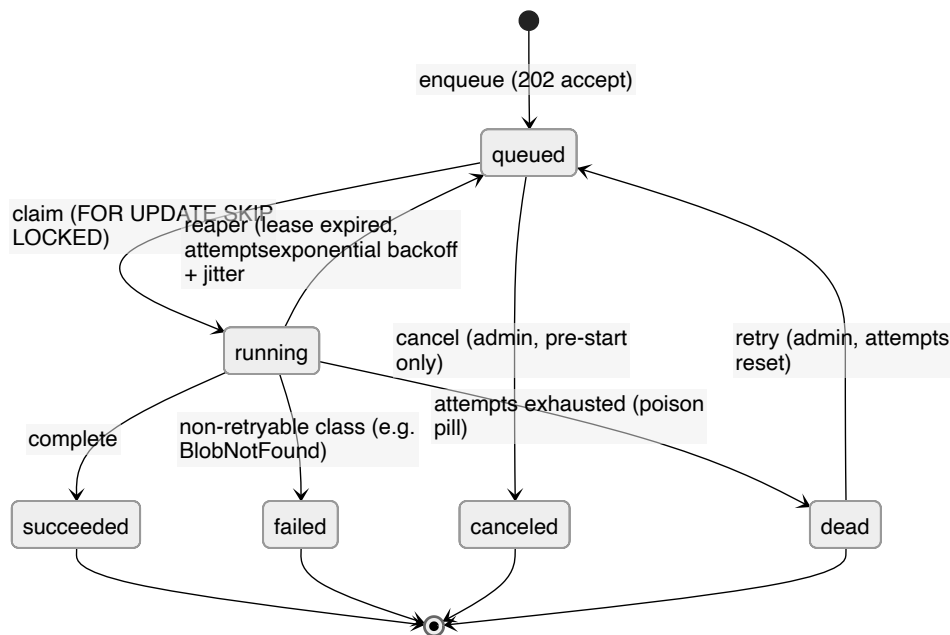




- `app.current_client_id` is bound per checkout in `di.db.acquire(client_id)`; the `tenant_isolation` policy (`004_rls.sql`) uses it in both `USING` and `WITH CHECK`. `FORCE ROW LEVEL SECURITY` means even the table owner is filtered.
- **Role split** (`006_roles_and_grants.sql`, `docker/initdb/01_roles.sql`):
 - `di_owner` — runs migrations (DDL); `NOSUPERUSER NOBYPASSRLS`.
 - `di_app_rw` — `NOLOGIN` group role holding explicit per-table DML grants (never `ALL TABLES`, so hash partitions get zero direct grants — closing the “`SELECT * FROM knode_p3`” bypass by construction).
 - `di_app` — the API login role, member of `di_app_rw`.
 - `di_worker` — `NOLOGIN` group role that holds the `worker_claim` permissive policy on `di_job` (`T0 di_worker USING(true)`), letting a worker claim cross-tenant *without* a GUC-trust hack.
 - `di_worker_login` — the worker login role, **member of both `di_app_rw` and `di_worker`** (`initdb/01_roles.sql:25`). This dual membership is why a **single-process deployment connects as `di_worker_login`**: it can do tenant-scoped pipeline writes *and* cross-tenant queue claims (see §9).
- The production **posture guard** (`di/posture.py`) verifies at runtime that the connected role is neither `rolsuper` nor `rolbypassrls`, that `tenant_isolation` exists on every tenant table, and that no partition has a leaked direct grant (`di/db.py:assert_rls_posture`).

6. The durable job queue & workers

A Postgres-native queue (no external broker) with lease/heartbeat/reaper semantics.



- **Claim** (`di/jobs.py:claim`): a two-level query — a window-function CTE selects cap-aware candidates per tenant (`row_number()` OVER (PARTITION BY `client_id` ...)) bounded by `ingest_tenant_max_running`, feeding an inner `FOR UPDATE SKIP LOCKED` re-check. (Two levels because Postgres forbids combining window functions with `FOR UPDATE` in one query level.) Ordering is global FIFO (`priority`, `run_after`, `created_at`, `id`).
- **Per-tenant running cap** is *hard*, enforced inside a single claim call (a 250k-document backfill for one tenant cannot starve the fleet).
- **Lease / heartbeat / reaper**: a claimed job has a lease (`job_lease_seconds`, default 300s), renewed at `lease/4`. A worker that dies stops heartbeating; the **reaper** (run by every worker, jittered) requeues expired-lease jobs with exponential backoff, or dead-letters those past `max_attempts`.
- **Fenced worker writes**: `heartbeat` / `complete` / `append_event` / `set_status` all filter `WHERE locked_by = $worker AND status = 'running'`. A zombie worker whose job was reclaimed writes zero rows and cancels its local task — it can never resurrect a job another worker now owns.
- **Unified terminal taxonomy**: attempts-exhausted **always** ends `dead` (the page-worthy `di_jobs_dead_total` alert); `failed` is reserved for non-retryable exception classes (e.g. `BlobNotFound`); `canceled` is a pre-start cancel (no preemption of running work).
- **arep job kind**: deferred accessibility-rep generation is a real queued job, idempotent per `version_id`.
- **Wake latency**: `LISTEN/NOTIFY` on `di_job_new` (dedicated non-pool connection) wakes an idle worker instantly; a `job_poll_interval_seconds` fallback covers a missed NOTIFY. Correctness never depends on NOTIFY.
- **Graceful drain**: `SIGTERM` → stop claiming → wait `job_drain_timeout_seconds` → release stragglers → exit 0.

Note on `blob_gc`: `di/config.py` carries a `dead_blob_retention_days` setting and a comment referencing a `di.tools.blob_gc` sweep / `enqueue(kind="blob_gc")`. That scheduled GC job kind is **documented future work, not yet implemented** — right-to-erasure for job-payload blobs is already covered today by `purge_client`'s tenant-prefix `BlobStore.delete_client()` sweep (content-addressed keys are tenant-prefixed). Treat the comment as a forward reference.

7. What makes it fast

The user-visible latencies that matter are (a) how quickly an upload is *accepted*, (b) how quickly a document becomes *queryable*, and (c) how quickly reads return at *millions-of-tenants* scale. Each is addressed by a specific, in-code mechanism.

7.1 OCR & INGEST LATENCY AVOIDANCE

Mechanism	How it works	Evidence	Impact
Non-blocking accept	The upload is persisted to the blob store and a <code>di_job</code> row is written, then <code>202</code> returns. OCR/LLM/embedding happen later on a worker.	<code>di/routers/ingest.py</code> (blob-put → <code>enqueue</code> → <code>202</code>)	The caller's request returns in ~blob-write time , not ~OCR time (tens of seconds for a scanned multi-page PDF).
Content-hash noop before OCR	<code>content_hash = sha256(bytes)</code> ; if the current version already has that hash and <code>ingest_complete</code> , the pipeline emits <code>done{noop}</code> and returns <i>before</i> calling OCR.	<code>di/pipeline.py</code> pre-OCR noop check; <code>di/store.py:create_version</code>	An unchanged re-upload skips the entire OCR + LLM + embedding cost — the single most expensive path — and completes in milliseconds .
OCR off the event loop	OCR (Azure Read polling / pdf2image / tesseract / pypdf) runs via <code>anyio.to_thread.run_sync</code> .	<code>di/pipeline.py</code> OCR stage	A slow OCR call never blocks the process's other concurrent ingests or reads.
Gate minimizes LLM egress	Only <code>SEND_TO_LLM</code> documents call the model gateway; everything sensitive or low-confidence stays <code>DETERMINISTIC_ONLY</code> (regex + checksums).	<code>di/gate/routing.py</code>	The costly LLM round-trip is <i>skipped entirely</i> for the majority of sensitive KYC docs — faster and cheaper, and PII-safe.
Async accessibility reps	With <code>AREP_ASYNC=true</code> , arep generation (5 rep types × N content nodes, each an LLM call) is a separate queue job, not inline.	<code>di/pipeline.py</code> arep branch; <code>di/worker.py:_run_arep</code>	The primary ingest job reaches <code>succeeded</code> without waiting on rep generation; reps land shortly after, decoupled.

7.2 READ & QUERY SPEED

Mechanism	How it works	Evidence	Impact
ltree GiST subtree fetch	A document / path subtree is a single <code>path</code> <code><@ \$prefix::ltree</code> range scan on a GiST index, not a recursive CTE.	<code>knode_path_gist (003:41);</code> <code>di/store.py:fetch_subtree,</code> <code>hybrid_search _scope</code>	Fetching a document's tree or a scoped branch is an index range scan , independent of total tree size.
pgvector HNSW ANN	Semantic search is <code>ORDER BY embedding</code> <code><=> \$query::vector LIMIT n</code> over an HNSW index (per partition).	<code>di/db.py:550-572 ;</code> <code>di/store.py:hybrid_search</code> vector legs	Approximate-nearest-neighbor in sub-linear time vs. a brute-force full scan.
Hybrid RRF (lexical + vector)	Four ranked legs — <code>knode.content_tsv</code> , <code>arep.rep_tsv</code> , <code>knode.content_embedding</code> , <code>arep.rep_embedding</code> — fused by Reciprocal Rank Fusion, then parent <code>knode</code> rows fetched by id.	<code>di/store.py:hybrid_search,</code> <code>_rrf</code>	Recall of both keyword and semantic matches without a re-ranker in the hot path; each leg is index-served and <code>LIMIT</code> -bounded (<code>pool_n = max(top_k*5, 50)</code>).
Generated tsvector + GIN	Full-text vectors are computed on write (<code>GENERATED ALWAYS AS ... STORED</code>) and GIN-indexed.	<code>003:20-21,42,59,66</code>	Lexical search is a GIN lookup; no per-query <code>to_tsvector</code> cost on the corpus.
Index-many / return-parent	Many searchable <code>arep</code> surfaces per chunk; a hit maps back to one canonical <code>knode</code> via <code>arep_client_knode</code> .	<code>arep_client_knode (003:67);</code> <code>hybrid_search arep</code> legs	High recall (a chunk is findable by question, paraphrase, or translation) without duplicating returned content.

7.3 CLIENT-LEVEL ACCESS & ISOLATION AT SCALE

Mechanism	How it works	Evidence	Impact
HASH partitioning by <code>client_id</code>	<code>knode</code> / <code>arep</code> are <code>PARTITION BY HASH (client_id)</code> into <code>PG_HASH_PARTITIONS</code> partitions (default 64). Every tenant query includes <code>client_id</code> , so the planner prunes to one partition .	<code>003:39,63</code> ; <code>di/db.py:_create_hash_partitions</code>	Per-tenant scan/index cost is bounded by <i>one partition's</i> size, not the whole corpus — the key to millions-of-tenants scale.
RLS predicate pushdown	The <code>tenant_isolation</code> policy predicate is ANDed into every plan; combined with partition pruning, a tenant only ever touches its own slice.	<code>004_rls.sql</code>	Isolation is <i>free</i> at query time (a predicate the planner already uses for pruning) and impossible to forget in app code.
Per-client partial & composite indexes	<code>di_documents_client</code> (partial <code>WHERE deleted_at IS NULL</code>), <code>knode_client_doc_ver</code> , <code>knode_attr</code> (partial <code>WHERE node_type='fact'</code>), <code>client_merged_fact_client</code> , etc.	<code>002</code> , <code>003</code>	Client-scoped list/lookup queries hit narrow, tenant-leading indexes.
Keyset (cursor) pagination	Document / job / change-feed / access-log listings page by <code>(created_at, id) < cursor</code> , not <code>OFFSET</code> .	<code>di/store.py:list_documents</code> , <code>fetch_access_log</code> ; <code>di/jobs.py</code> cursor codec	Page <i>N</i> costs the same as page 1 — no growing <code>OFFSET</code> scan.
Merged-fact “current view”	Client-level answers read <code>client_merged_fact</code> (one row per attribute/instance) instead of re-aggregating <code>knode</code> facts across documents at query time.	<code>002:62–76</code> ; <code>di/store.py:fetch_merged_facts</code>	“What is this client’s current passport number?” is a single indexed row read.

7.4 WRITE / INGEST THROUGHPUT

Mechanism	How it works	Evidence	Impact
FOR UPDATE SKIP LOCKED queue	Workers claim without blocking each other; contention shrinks the batch harmlessly instead of serializing.	<code>di/jobs.py:claim</code>	Near-linear scaling as workers are added (<code>--scale worker=N</code>).
Partial claim/lease indexes	<code>di_job_claim</code> (WHERE status='queued') and <code>di_job_lease</code> (WHERE status='running') keep the hot scans tiny regardless of table size.	<code>010_job_queue.sql</code>	Claim and reaper stay fast even with millions of terminal rows.
Per-tenant fairness cap	The window-function claim bounds any one tenant's share of fleet concurrency.	<code>di/jobs.py:claim</code> ; <code>ingest_tenant_max_running</code>	A giant single-tenant backfill can't starve everyone else.
HOT-update tuning on <code>di_job</code>	<code>fillfactor=70</code> + aggressive autovacuum thresholds for the high-churn status/heartbeat updates.	<code>010_job_queue.sql</code>	Status flips and heartbeats stay HOT (on-page), reducing bloat and index churn.

8. API surface (reference)

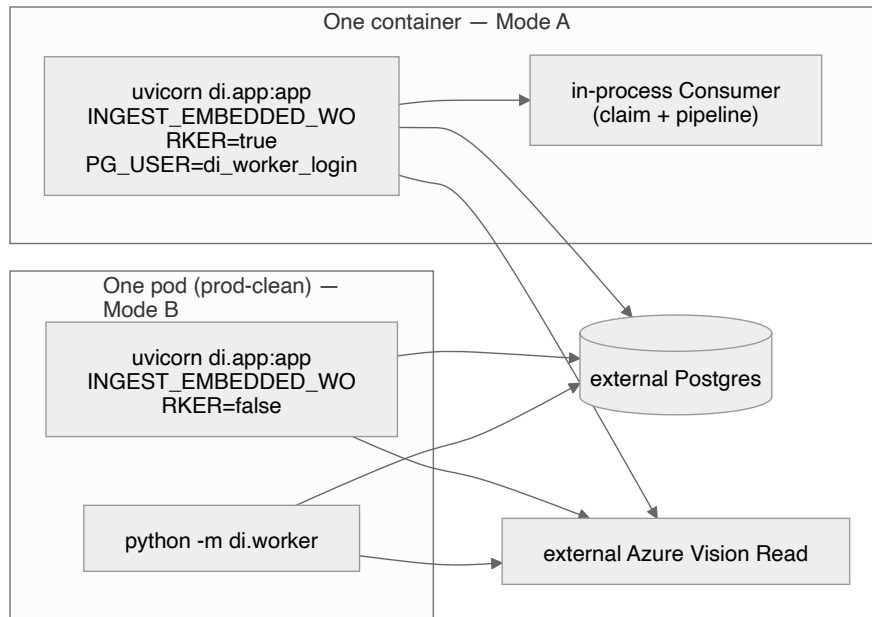
All routes are under `/api/v1`, authenticated by `X-API-Key`, and authorized to the path/argument `client_id`. Masking defaults to the server-side policy (`MASK_BY_DEFAULT`).

Method & path	Scope	Purpose
POST /ingest	ingest	Accept a document → 202 + job_id (or SSE stream with ?stream=true)
GET /jobs	read	List a client's ingest jobs (keyset paginated)
GET /jobs/{id}	read	Poll one job: status, stage events, error
POST /jobs/{id}/retry	admin	Requeue a dead job (attempts reset)
POST /jobs/{id}/cancel	admin	Cancel a queued job (no preemption)
GET /clients/{id}/tree	read	Nested knowledge subtree (doc/path scoped)
GET /clients/{id}/facts	read	Merged client facts (verified_only, multi-valued instance_key)
GET /clients/{id}/documents	read	List documents (keyset; excludes raw OCR text)
GET /clients/{id}/changes	read	Version change feed (monotonic after_seq cursor)
GET /clients/{id}/docs/{doc}/manifest	read	What a document can answer, and how
GET /clients/{id}/docs/{doc}/answerable	read	Questions derived from a doc's accessibility reps
POST /clients/{id}/search	read	Hybrid (lexical + vector) search, grounded in sources
GET /nodes/{id}/provenance	read	Trace a node to source doc/page/bbox/extractor/model
POST /admin/clients/{id}/adjudicate	admin	Reviewer verdict on a fact / instance (survives re-merge)
GET/DELETE /admin/clients/{id}/adjudications[/history]	admin	Live verdicts, append-only history, clear a verdict
DELETE /admin/clients/{id}/documents/{doc}	admin	Hard-delete a document + re-merge
POST /admin/clients/{id}/purge	admin	Tenant off-boarding / right-to-erasure
GET/POST/DELETE /admin/keys[...], .../rotate	admin	API key lifecycle (create/list/revoke/rotate)
GET/PUT /admin/tenants/{id}/policy	admin	Per-tenant ingest-quota overrides
GET /admin/access-log	admin	"Who read this client's data?" (append-only audit)
GET /health · /readyz · /metrics	—	Liveness · readiness · Prometheus

9. Running everything in one container / pod

Requirement: run *all application components* (API + ingest worker + migrations) in a single container/pod, with **Postgres** and **Azure Vision Read** as **external services** reached purely via env vars.

This is already supported. The app lifespan (`di/app.py:279–284`) starts an in-process worker `Consumer` when `INGEST_EMBEDDED_WORKER=true`, and migrations run inside the same startup (`MIGRATIONS_MODE=auto`). There are two clean ways to satisfy “one pod”, with an honest trade-off between them.



MODE A — SINGLE PROCESS, EMBEDDED WORKER (SIMPLEST)

One `uvicorn di.app:app` process is the API *and* the worker *and* runs migrations at startup. Best for **dev, demo, single-tenant, and low/medium volume**.

Crucial detail: connect as `di_worker_login`. That role is a member of *both* `di_app_rw` (tenant-scoped pipeline writes) *and* `di_worker` (cross-tenant queue claims via the `worker_claim` RLS policy) — `docker/initdb/01_roles.sql:25`. `di_app` alone cannot claim; `di_owner` alone cannot claim (not a `di_worker` member). One process doing both jobs needs the one role that spans both groups.

```

docker run --rm -p 8080:8080 \
  -e DI_ENV=local \
  -e INGEST_EMBEDDED_WORKER=true \
  -e MIGRATIONS_MODE=auto \
  -e RLS_ENABLED=true \
  -e PG_HOST=<org-postgres-host> -e PG_PORT=5432 \
  -e PG_DATABASE=document_intelligence -e PG_SCHEMA=di \
  -e PG_USER=di_worker_login -e PG_PASSWORD=<secret> \
  -e PG_MIGRATION_USER=di_owner -e PG_MIGRATION_PASSWORD=<secret> \
  -e PG_HASH_PARTITIONS=64 \
  -e AZURE_VISION_ENDPOINT=<org-azure-read-endpoint> \
  -e AZURE_VISION_KEY=<secret> \
  -e RETRIEVAL_BASE_URL=<org-model-gateway> -e RETRIEVAL_API_KEY=<secret> \
  -e BLOB_BACKEND=postgres \
  -e DI_BOOTSTRAP_API_KEY=<first-key-or-unset-and-mint> \
  document-intelligence:latest
  
```

MODE B — TWO PROCESSES, ONE POD (PRODUCTION-POSTURE-CLEAN)

Run `uvicorn` (API, `INGEST_EMBEDDED_WORKER=false`) **and** `python -m di.worker` **side by side in the same pod/container**. In Kubernetes this is the idiomatic “one Pod, two containers”; in a literal single Docker container it’s a tiny supervisor/entrypoint that launches both. This keeps the OCR/CPU-heavy worker off the API’s event loop while still being one deployable unit.

Why this exists — the honest trade-off: the production posture guard (`di/posture.py`) **refuses to boot** when `INGEST_EMBEDDED_WORKER=true` and `DI_ENV ∈ {staging, prod, production}`. That is deliberate: embedding the worker couples OCR/LLM CPU spikes to API tail latency, the exact coupling the durable-queue upgrade removed. So:

- **Mode A in production** would require lowering `DI_ENV` to a non-prod value, which *also* disables every other prod guard (RLS check, auth requirement, strict audit). **Don't** do that to get a single process.
- **Mode B** keeps `INGEST_EMBEDDED_WORKER=false`, so the guard passes and every prod protection stays on — while still being **one pod**. This is the recommended single-pod *production* topology.

Small addition Mode B needs (literal single container): a supervisor entrypoint that runs both processes and forwards SIGTERM to each for graceful drain. A `di.migrate` init step (or `MIGRATIONS_MODE=verify` + a migration init-container) runs the schema once. In k8s, prefer two containers in one Pod + an init-container for migrations — no new code required.

REQUIRED / NOTABLE ENVIRONMENT VARIABLES

Variable	Purpose	Example
<code>PG_HOST</code> / <code>PG_PORT</code> / <code>PG_DATABASE</code> / <code>PG_SCHEMA</code>	External Postgres	<code>pg.internal</code> / <code>5432</code> / <code>document_intelligence</code> / <code>di</code>
<code>PG_USER</code> / <code>PG_PASSWORD</code>	Runtime role — <code>di_worker_login</code> for Mode A; <code>di_app</code> for the API in Mode B	<code>di_worker_login</code>
<code>PG_WORKER_USER</code> / <code>PG_WORKER_PASSWORD</code>	Worker-pool role (Mode B worker, or leave unset in Mode A to reuse <code>PG_USER</code>)	<code>di_worker_login</code>
<code>PG_MIGRATION_USER</code> / <code>PG_MIGRATION_PASSWORD</code>	Owner role for DDL (<code>MIGRATIONS_MODE=auto</code> / <code>di.migrate</code>)	<code>di_owner</code>
<code>PG_HASH_PARTITIONS</code>	Partition count — must match the value the DB was first initialized with	<code>64</code>
<code>MIGRATIONS_MODE</code>	<code>auto</code> (apply in-process) / <code>verify</code> (assert only) / <code>off</code>	<code>auto</code>
<code>RLS_ENABLED</code>	Keep <code>true</code> everywhere holding real data	<code>true</code>
<code>INGEST_EMBEDDED_WORKER</code>	<code>true</code> = Mode A; <code>false</code> = Mode B / scaled	<code>true</code>
<code>AZURE_VISION_ENDPOINT</code> / <code>AZURE_VISION_KEY</code>	External OCR	<code>https://...cognitiveservices.azure.com/</code>
<code>RETRIEVAL_BASE_URL</code> / <code>RETRIEVAL_API_KEY</code>	External model gateway (or <code>DI_RETRIEVAL_STUB=true</code>)	<code>https://gateway.internal</code>
<code>BLOB_BACKEND</code> (+ <code>S3_*</code>)	<code>postgres</code> (single backup domain) / <code>s3</code> / <code>local</code>	<code>postgres</code>
<code>DI_ENV</code>	<code>local</code> / <code>dev</code> (guards off) vs <code>staging</code> / <code>prod</code> (guards on)	<code>prod</code>
<code>DI_BOOTSTRAP_API_KEY</code>	Seeds a first key (unset in prod; mint via <code>python -m di.tools.keys</code>)	(unset in prod)
<code>INSTANCE_FINGERPRINT_HMAC_KEY</code>	Salts multi-valued-fact fingerprints (required in prod)	(secret)

VERIFICATION (ONCE THE CONTAINER IS UP)

1. `GET /health` → `200 {status: ok}`.

2. `GET /readyz` → `ready: true`; the `queue` component reports `embedded_worker: true` (Mode A) or `false` (Mode B), `depth: 0`.
 3. `POST /api/v1/ingest` a small document → `202 {job_id}`.
 4. `GET /api/v1/jobs/{job_id}` poll → `succeeded`, with `ocr → gate → extract → subtree → merge → done` stage events. In Mode A this proves one process did API **and** worker **and** pipeline.
 5. `GET /metrics` → `di_queue_depth`, `di_jobs_inflight`, `di_jobs_claimed_total` present.
-

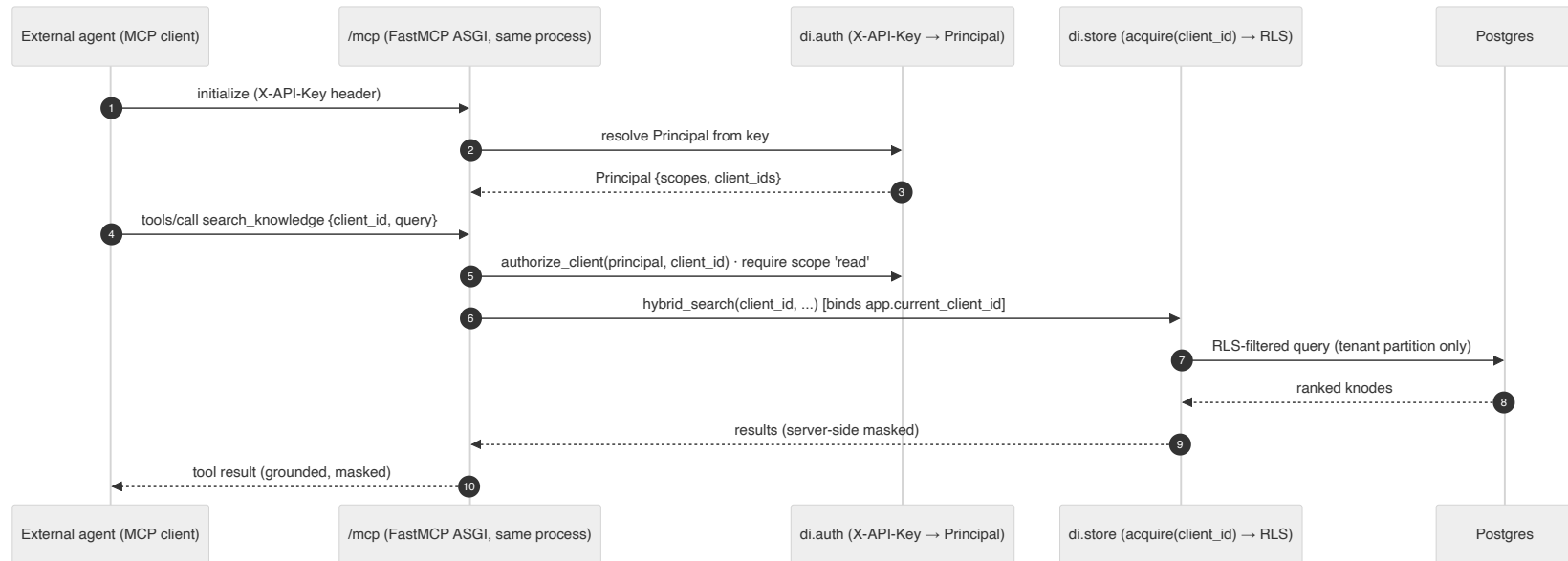
10. MCP endpoint — design (for agents to use the platform)

Goal: expose the platform's capabilities over the **Model Context Protocol** so other AI agents can search a client's knowledge, read facts and provenance, submit ingests, and poll jobs — **inside the same single container**, reusing the exact same auth and per-tenant RLS as the REST API, with zero new isolation surface.

Status: implemented and live-verified. The endpoint ships in `di/mcp/` and mounts at `/mcp` on the same app/container. It was verified end-to-end over the real MCP streamable-HTTP transport: tool discovery, `X-API-KEY` auth (good/bad/missing key), the per-tenant RLS boundary (a tenant-scoped key is refused another tenant), and a full ingest round-trip (submit → poll → succeeded → facts/search) through the embedded worker. See "Implementation" below.

10.1 SHAPE & TRANSPORT

- **Transport: Streamable HTTP** (the modern MCP HTTP transport), served by the **official Python MCP SDK** (`mcp`, which bundles `FastMCP`). `FastMCP` produces an **ASGI app** that mounts onto the existing FastAPI app — so it runs **in the same process and container** as the API and (in Mode A) the embedded worker. No second port, no second deployable.
- **Mount point:** `app.mount("/mcp", mcp_server.streamable_http_app())` in `di/app.py:create_app()`, after the routers. The `FastMCP` server is built with `streamable_http_path="/"` so the mount point *is* the full path, and an explicit `/mcp` → `/mcp/` 307 redirect is registered ahead of the SPA catch-all so agents can point at either `/mcp` or `/mcp/`. The streamable-HTTP session manager is started from the parent app's lifespan (`async with app.state.mcp.session_manager.run(): yield`) because Starlette does not run a mounted sub-app's own lifespan. Agents connect to `https://<host>/mcp`.



10.2 AUTH & TENANT SCOPING (IDENTICAL GUARANTEES TO REST)

- **Authentication:** the MCP client sends the **same** `X-API-Key` (as an HTTP header on the streamable-HTTP connection). An ASGI middleware in front of the MCP app resolves it to a `Principal` exactly as the REST middleware does — **one auth code path, no parallel auth to keep in sync**.
- **Tenant scoping:** every tool takes `client_id` as an argument and calls `authorize_client(principal, client_id)` + the tool's required scope *before* touching data. Every DB read still goes through `di.store` → `acquire(client_id)`, which binds `app.current_client_id`, so **RLS is preserved unchanged** — an MCP caller cannot reach another tenant any more than a REST caller can.
- **Masking:** tool results use the same server-side `serving.project_*` masking projection (`MASK_BY_DEFAULT`), so sensitive values are redacted by default over MCP too.
- **Rate limiting & audit:** the MCP mount sits behind the same middleware chain, so per-key rate limiting and the read-side access audit apply to MCP calls as well.

10.3 PROPOSED TOOLS

MCP tool	Scope	Wraps	Input	Output
<code>search_knowledge</code>	<code>read</code>	<code>store.hybrid_search</code> + <code>serving.project_nodes</code>	<code>client_id</code> , <code>query</code> , <code>top_k?</code> , <code>scope_path?</code> , <code>doc_id?</code> , <code>mask?</code>	Ranked, grounded knode hits
<code>get_client_facts</code>	<code>read</code>	<code>store.fetch_merged_facts</code> + <code>serving.project_facts</code>	<code>client_id</code> , <code>attribute_key?</code> , <code>verified_only?</code> , <code>mask?</code>	Merged facts (multi-valued aware)
<code>get_document_tree</code>	<code>read</code>	<code>store.fetch_subtree</code> + <code>serving.nest_tree</code>	<code>client_id</code> , <code>doc_id?</code> , <code>path?</code> , <code>max_depth?</code> , <code>mask?</code>	Nested knowledge subtree
<code>list_documents</code>	<code>read</code>	<code>store.list_documents</code>	<code>client_id</code> , <code>cursor?</code> , <code>limit?</code>	Document summaries (no raw OCR text)
<code>get_document_manifest</code>	<code>read</code>	<code>serving.build_manifest</code> / <code>answerable_questions</code>	<code>client_id</code> , <code>doc_id</code>	What the doc can answer
<code>get_node_provenance</code>	<code>read</code>	<code>store.fetch_node</code>	<code>client_id</code> , <code>node_id</code>	Source doc/page/bbox/extractor/model
<code>submit_ingest</code>	<code>ingest</code>	<code>jobs.enqueue</code> (+ blob-at-accept)	<code>client_id</code> , <code>filename</code> , <code>content(base64)</code> , <code>mime?</code> , <code>idempotency_key?</code>	<code>{ job_id, status }</code>
<code>get_job_status</code>	<code>read</code>	<code>jobs.get_job</code>	<code>client_id</code> , <code>job_id</code>	Status + stage events + error

Admin-scoped tools (`adjudicate`, `purge_client`, key management) are intentionally **excluded from the default MCP surface** — destructive/irreversible tenant operations should not be one agent tool-call away. They can be added behind an explicit `admin`-scoped, separately-gated MCP server if a workflow genuinely needs them.

10.4 RESOURCES

MCP **resources** (read-only, addressable context) complement the tools:

- `di://clients/{client_id}/facts` — the merged fact sheet as a resource an agent can attach as context.
- `di://clients/{client_id}/documents/{doc_id}/manifest` — a document’s answerable-questions manifest.
- `di://clients/{client_id}/documents` — the document index.

(Resources are the natural fit for “load this client’s current state into my context”; tools are the fit for search/actions.)

10.5 MOUNTING SO “ONE CONTAINER” STILL HOLDS

Because FastMCP is mounted as an ASGI sub-app on the existing FastAPI app, the MCP endpoint ships in the **same image and same process** as the API. In **Mode A** (embedded worker) a single `uvicorn di.app:app` process then serves **API + MCP + ingest worker** together — literally all components in one container. In **Mode B** the MCP endpoint rides with the API process; the worker stays separate but in the same pod.

10.6 IMPLEMENTATION (SHIPPED)

File	Purpose	Status
<code>di/mcp/__init__.py</code>	Package + <code>build_mcp</code> export	✓
<code>di/mcp/server.py</code>	<code>build_mcp()</code> — constructs the FastMCP server (stateless, JSON responses), registers the 9 tools above, each delegating to <code>di.store</code> / <code>di.jobs</code> / <code>di.serving</code> under <code>mcp_auth.require</code> (auth + scope + tenant)	✓
<code>di/mcp/auth.py</code>	<code>authenticate</code> / <code>authorize</code> / <code>require</code> — resolves <code>X-API-KEY</code> from <code>ctx.request_context.request</code> via <code>di.auth.resolve_principal</code> (the one shared key path), enforces scope + <code>client_id</code> , applies the per-key rate-limit backstop	✓
<code>di/app.py</code> (edit)	Mounts <code>/mcp</code> (<code>_maybe_mount_mcp</code> , fail-open), the <code>/mcp</code> → <code>/mcp/</code> redirect, the lifespan <code>session_manager.run()</code> , and a <code>mcp</code> readiness component	✓
<code>di/config.py</code> (edit)	<code>mcp_enabled: bool = True</code>	✓
<code>pyproject.toml</code> (edit)	<code>mcp>=1.0</code> added to core dependencies (so the Docker image installs it)	✓
<code>tests/test_mcp.py</code>	10 unit tests: auth (missing/bad/good/disabled), authorize (scope/tenant/wildcard), tool registry present, destructive tools absent	✓

Verification. 507 unit tests pass (ruff + mypy clean). Live end-to-end against a running instance (real MCP client over streamable-HTTP): 9 tools discovered / no destructive tools; `submit_ingest` → `get_job_status` polled to `succeeded` through the embedded worker; `get_client_facts` (masked) and `search_knowledge` return grounded, tenant-scoped results; idempotent re-submit reuses the job; a bad/missing key fails closed as a tool error; and a **tenant-scoped key reads its own tenant but is refused another tenant** — the same RLS boundary as REST.

Deferred (documented follow-ups): MCP *resources* (§10.4) — the tool surface fully covers the “use the services” need, and resources add a second auth path that warrants its own pass; recording MCP tool calls in the read-side access audit (the middleware resolves `client_id` from path/query, not the JSON-RPC body — a per-tool audit hook would close this).

10.7 SECURITY CONSIDERATIONS

- **No isolation bypass:** every tool goes through `authorize_client` + `acquire(client_id)`; there is no path to data that skips RLS. This must be covered by an explicit cross-tenant-denial test.
- **Scope enforcement:** `read` vs `ingest` scopes mirror REST; admin operations are excluded by default.

- **Masking on by default:** an agent gets redacted sensitive values unless the key is cleared and the caller opts out — same policy as REST.
- **submit_ingest size limits:** enforce `max_upload_mb` and the per-tenant admission quota exactly as the REST accept path does (reuse `_require_durable_blob_backend`, `_enforce_ingest_quota`).
- **Audit:** MCP tool calls that resolve a `client_id` are recorded by the same read-side access audit.

10.8 DECISIONS MADE (CHANGEABLE ON REQUEST)

The shipped implementation took the recommended default for each original open question; each is straightforward to revisit:

1. **Auth mechanism** — reuses the existing `X-API-KEY` header (zero new surface, one shared key path). MCP-native OAuth remains an option if an agent platform requires it.
2. **Admin tools** — excluded from the MCP surface. A separate, explicitly-gated `admin` MCP server could expose them if a workflow genuinely needs it.
3. **Ingest progress** — poll-based via `get_job_status` (matches the REST 202+poll contract). MCP progress-notification streaming for `submit_ingest` is a possible enhancement.

11. Appendix

11.1 MIGRATIONS

File	What it establishes
<code>001_extensions.sql</code>	<code>ltree</code> , <code>pgcrypto</code> (pgvector installed by the superuser bootstrap)
<code>002_core_tables.sql</code>	<code>di_documents</code> , <code>doc_version</code> , <code>di_entity</code> , <code>client_merged_fact</code> , <code>di_decision_trace</code>
<code>003_knode_arep.sql</code>	<code>knode</code> + <code>arep</code> (HASH-partitioned, GiST/GIN indexes, generated tsvectors)
<code>004_rls.sql</code>	<code>FORCE ROW LEVEL SECURITY</code> + <code>tenant_isolation</code> on every tenant table
<code>005_hardening.sql</code>	<code>di_job</code> , <code>di_blob</code> , change-seq sequence, additional indexes
<code>006_roles_and_grants.sql</code>	<code>di_owner</code> / <code>di_app_rw</code> / <code>di_worker</code> role split, explicit grants, <code>worker_claim</code> policy
<code>007_auth_hardening.sql</code>	<code>di_api_key</code> , <code>di_tenant_policy</code> , partitioned append-only <code>di_access_log</code>
<code>008_multi_valued_facts.sql</code>	<code>instance_key</code> + <code>di_fact_adjudication</code> ; <code>UNIQUE (client_id, attribute_key, instance_key)</code>
<code>010_job_queue.sql</code>	Durable-queue columns on <code>di_job</code> , partial claim/lease indexes, <code>doc_version.ingest_complete</code>
<code>011_doc_version_unique.sql</code>	<code>UNIQUE (client_id, doc_id, version_no)</code> backstop (shipped one release after 010's lock/retry code)

11.2 SELECTED CONFIGURATION (DI/CONFIG.PY)

```
DI_ENV, PG_*, PG_MIGRATION_*, PG_WORKER_*, MIGRATIONS_MODE, RLS_ENABLED, PG_HASH_PARTITIONS,
AZURE_VISION_*, RETRIEVAL_* / DI_RETRIEVAL_STUB, AUTH_ENABLED, DI_BOOTSTRAP_API_KEY, MASK_BY_DEFAULT,
GATE_DEFAULT_OPEN, AREP_ASYNC, INGEST_EMBEDDED_WORKER, JOB_LEASE_SECONDS, JOB_MAX_ATTEMPTS,
INGEST_TENANT_MAX_RUNNING, INGEST_MAX_ACTIVE_JOBS_PER_CLIENT, BLOB_BACKEND (+ S3_*), ACCESS_AUDIT_ENABLED /
ACCESS_AUDIT_STRICT, INSTANCE_FINGERPRINT_HMAC_KEY.
```

11.3 PRODUCTION POSTURE (ENFORCED AT BOOT BY DI/POSTURE.PY)

`RLS_ENABLED=true` · `AUTH_ENABLED=true` · `MASK_BY_DEFAULT=true` · `MIGRATIONS_MODE=verify` (or a distinct migration user) · `ACCESS_AUDIT_ENABLED` + `ACCESS_AUDIT_STRICT=true` · `INGEST_EMBEDDED_WORKER=false` · `BLOB_BACKEND` ∈ {`postgres`, `s3`} · `INSTANCE_FINGERPRINT_HMAC_KEY` set · `DI_BOOTSTRAP_API_KEY` unset (or strong). A production instance that violates any of these **refuses to start**. Runtime RLS facts (non-superuser role, live policies, no partition-grant leak) are additionally verified against the live database (`di/db.py:assert_rls_posture`).

Generated from the current codebase (`~/document_intelligence` , migrations 001–011). File/line references point at the source of each claim. The single-container role detail (§9, `di_worker_login`) and the MCP mount approach (§10) are verified against `docker/initdb/01_roles.sql` and `di/app.py` respectively; the MCP endpoint itself is a design to be implemented.